

Automatic testing of web a11y

From zero level to dev focused



Łukasz Nowak

Senior frontend dev



Łukasz Nowak



LukaszNowakPL

Current state of accessibility in web development

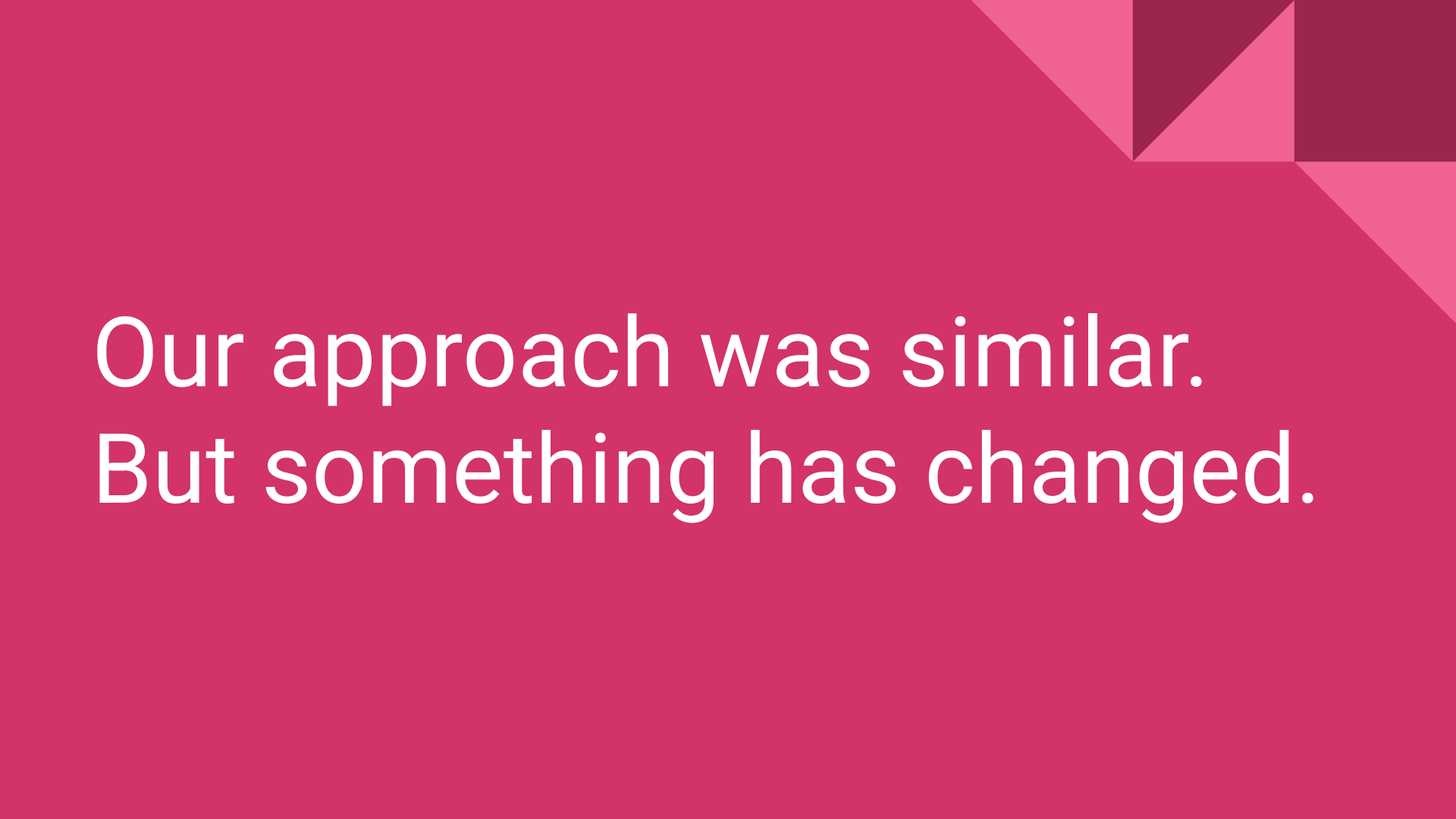
- Accessibility topics are common on frontend conferences / meetups
- June 28th, 2025 - European Accessibility Act requires service compliance at WCAG 2.1 AA level ⁽¹⁾
- 88% of websites are not fully compliant with latest standards ⁽²⁾⁽³⁾
- Because we all are having excuses

(1) <https://www.wcag.com/compliance/european-accessibility-act/>

(2) <https://www.accessibilitychecker.org/research-papers/the-state-of-web-accessibility-in-2024-research-report/>

(3) <https://webaim.org/projects/million/>





Our approach was similar.
But something has changed.



<https://www.youtube.com/watch?v=L0PRzIyP1Zc>

Auditing Design Systems for Accessibility by Anna E. Cook



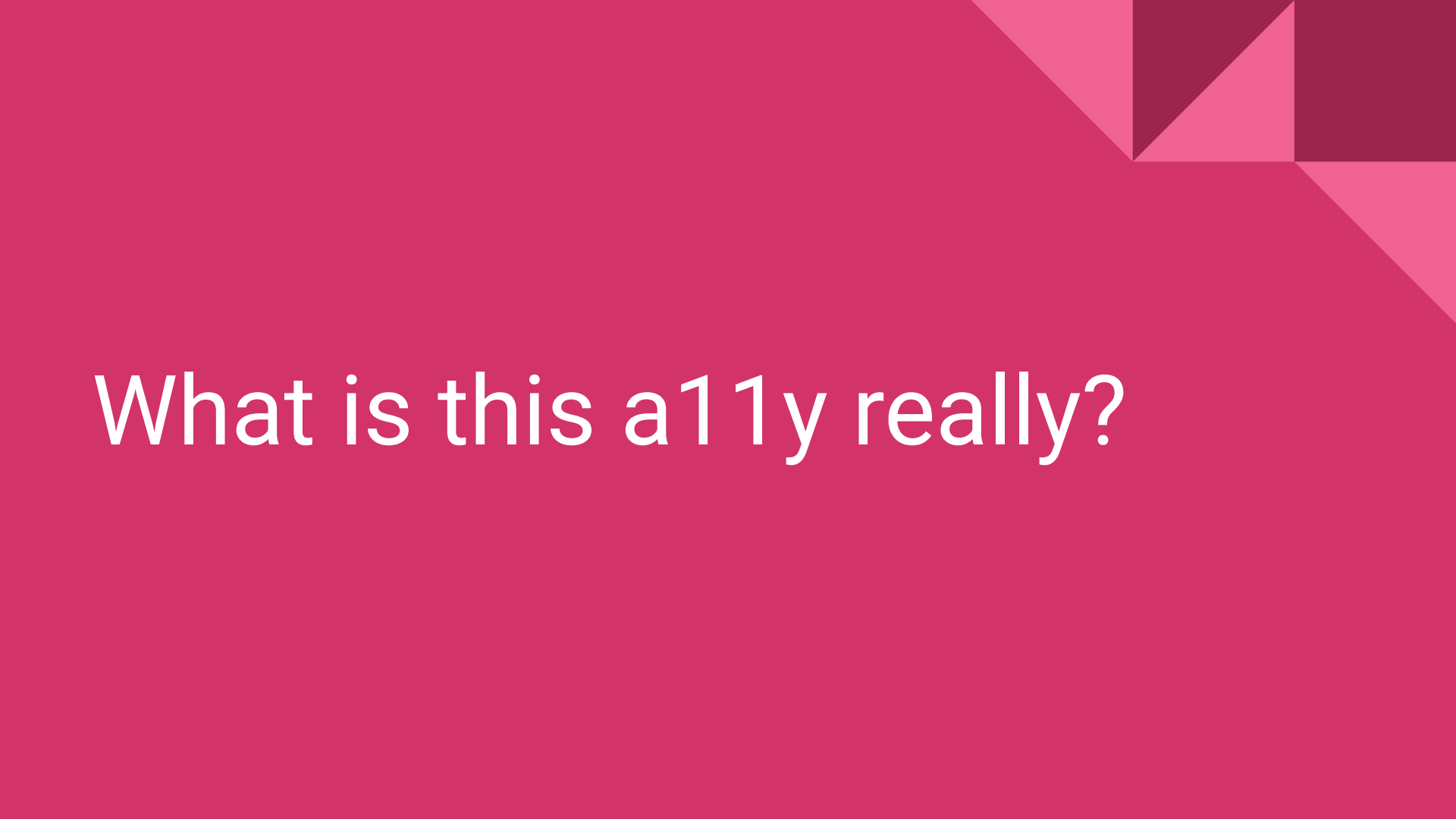
“Accessibility is not about being perfect. It’s about trying.”

Anna E. Cook, Wey Wey Web 2023



“It’s best to try a11y testing on new applications. You start small and grow as the project grows.”

Source unknown



What is this a11y really?

A11y is required for users experiencing any of ⁽¹⁾

- Visual Impairment
- Hearing Impairment
- Cognitive and Learning Disabilities
- Motor Impairment
- Speech Impairment
- Neurological Limitations

(1) <https://reciteme.com/news/common-website-accessibility-issues/>



Most common web a11y issues

- Colour contrasts - to cover by design
- Complicated content, structure or user journey - to cover by UX
- Poor element identification - this one is ours
- Problematic interaction - this one too



Poor element identification

- Vast possibilities of page presentation
 - Visual presentation (screen)
 - Audio description (screen reader)
 - Via dedicated assistive technologies (i.e. braille reader)
- No/poor description for items like images, links or buttons
- It's all about quality of HTML produced



Problematic interaction

- Interacting with the element on the page is:
 - Selecting the element
 - By searching on the page (i.e. `getByRole('button', {name: 'Send'})`)
 - By using elements list (navigation via Tab key)
 - Performing dedicated action (type, click)
- It's all about quality of HTML produced



How to approach automatic testing by devs?

- Semantic and well described HTML is the key
- No need to test using all assistive technologies
- It's all about HTML - we should test production bundle
- Playwright to be the tool of our choice
(however, it's not obligatory)



Example repository



Add airport

Name

IATA Code

Country

 ▾

Regions

- ☐ Aegean Islands
- ☐ Corfu Island (Kerkyra)
- ☐ Ha Long Island
- ☐ Rhodes Island

- ☒ Balearic Islands
- ☐ Crete Island
- ☐ Ionian Islands
- ☐ Tenerife Island

- ☐ Canary Islands
- ☐ Dodecanese Islands
- ☒ Maiorca Island

Vaccination Notes

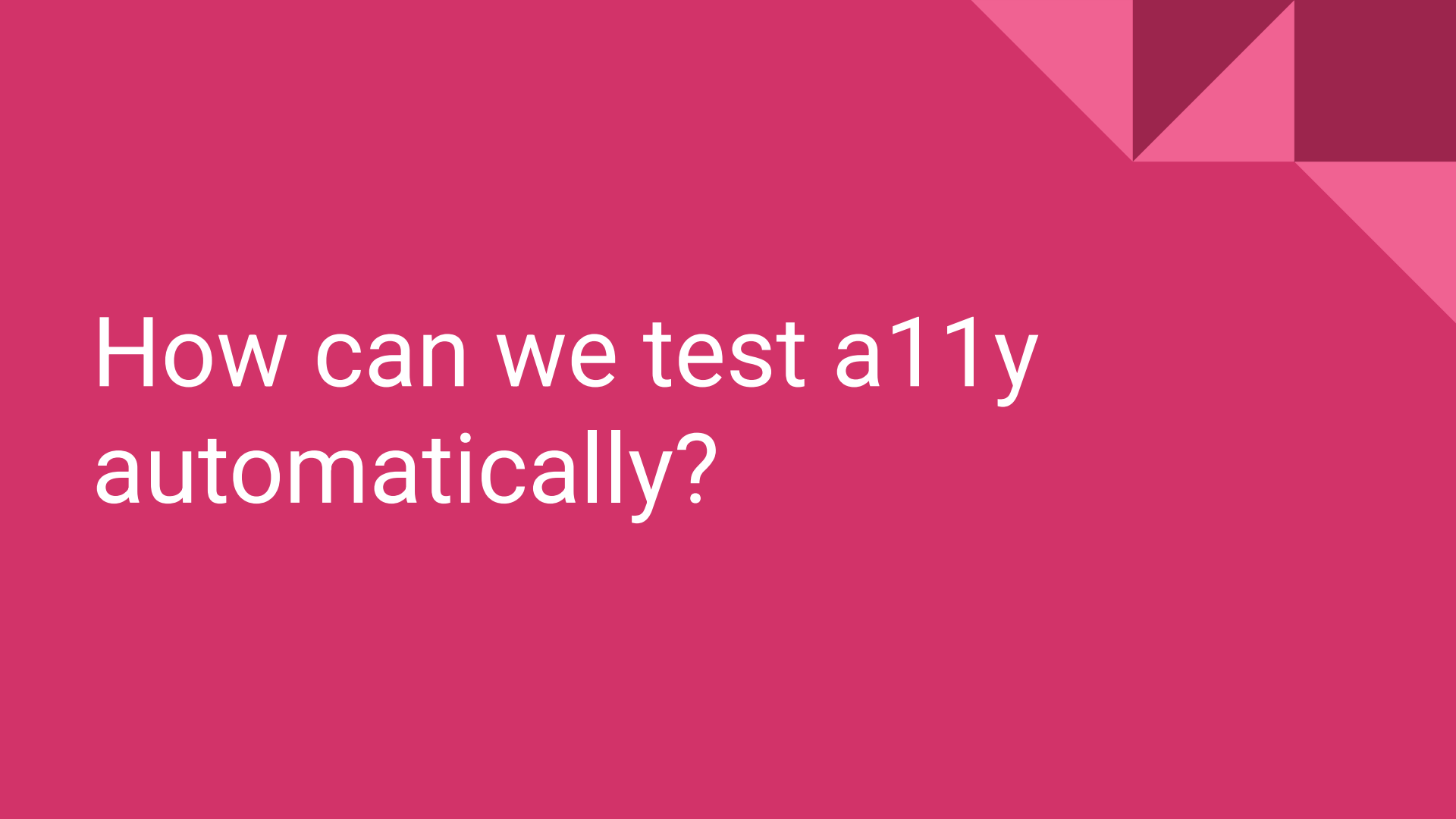
Example repository

- Fully functional basis of React SPA + Backend
- Reliable test examples
- Framework and reusables to run tests
- Note: The app is not a11y compliant!
(But you will find it out on test report)



<https://github.com/LukaszNowakPL/web-a11y-testing>





How can we test a11y
automatically?

User journey with keyboard navigation

- It's just another functional test - a low hanging fruit
- Minimal approach: cover at least a simplest happy path user journey



Test case actions

```
// Entering the page
public async proceedThroughPage(airport: AirportModel) {
    await this.assertNoFocus();
    await this.navigateToNextElement(); // ...

    // Filling airport name field
    await this.assertFocusedElement(this.page.getByRole('textbox', {name: /name/i}));
    await this.fill(airport.name);
    await this.navigateToNextElement(); // ...

    // Clicking submit button
    await this.assertFocusedElement(this.page.getByRole('button', {name: /send/i}));
    await this.performAction();
}
// Asserting data sent to backend service
```

Reusable helpers

```
private getFocusedElement () {  
    return this.page.locator('*:focus') as NotClickableLocator;  
}  
  
public async assertFocusedElement (expectedElement: Locator) {  
    expect(await this.getOuterHtml(this.getFocusedElement())).toStrictEqual(await this.getOuterHtml(expectedElement));  
}  
  
private async getOuterHtml (element: Locator) {  
    return await element.evaluate ((node) => node.outerHTML);  
}  
  
public async performAction () {  
    await this.page.press('body', this.KEYS.action);  
}  
  
public async selectFocusedElement () {  
    await this.page.press('body', this.KEYS.selection);  
}  
  
public async navigateToNextElement () {  
    await this.page.press('body', this.KEYS.navigateToNextElement);  
}
```

Static HTML audits

Same page, different states

Provide piece of data

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Piece of data

Submit

Provide piece of data

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Piece of data

This data is obligatory

Submit

Provide piece of data

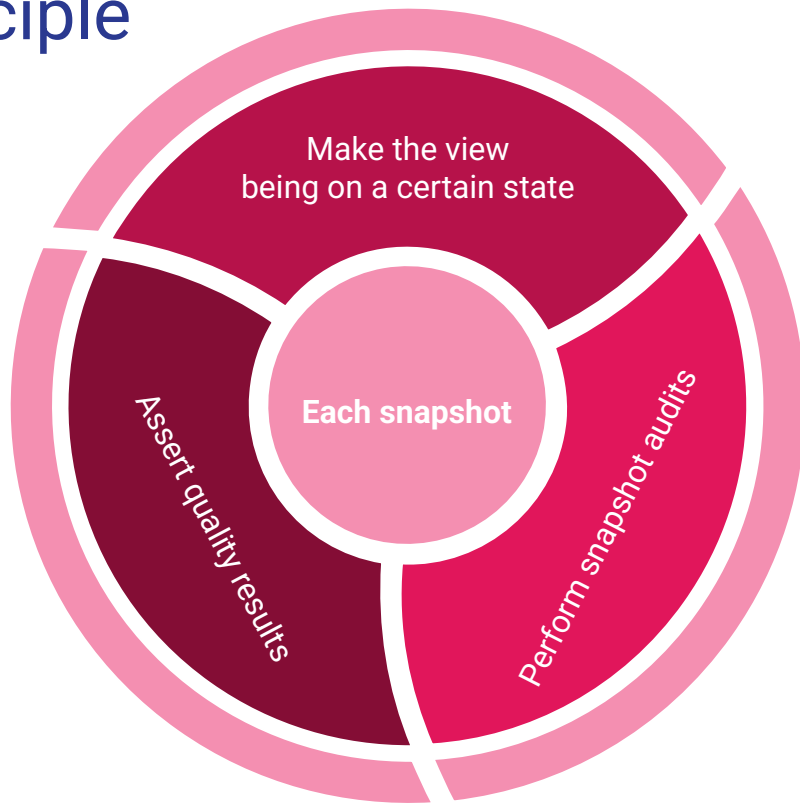
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Piece of data

Data required

Submit

Approach principle



Test case actions

```
// Then I wait until the page is ready  
await addAirportPage.assertReady();  
  
// When I perform snapshot analysis for initial view  
await addAirportPage.performSnapshotAnalysis('Initial view', {takeLighthouseSnapshot});  
  
// And I fulfill the form with correct data  
await addAirportPage.fulfillForm(airport);  
  
// Then I can see the form data is valid and Submit button is enabled  
await addAirportPage.assertSubmitEnabled();  
  
// When I perform snapshot analysis for provided data  
await addAirportPage.performSnapshotAnalysis('Form filled', {takeLighthouseSnapshot});  
// ...
```

The audit process

```
public async assertSnapshots(snapshotTitle: string, config?: AssertSnapshotConfig) {  
    await Promise.all([  
        this.assertAxeChecks(snapshotTitle, Boolean(config?.skipAxeViolationChecks)),  
        this.performAriaSnapshotComparison(snapshotTitle),  
        config?.takeLighthouseSnapshot !== undefined  
            && this.assertLighthouseSnapshot (snapshotTitle,  
config.takeLighthouseSnapshot ),  
    ]);  
}
```

Lighthouse audit

- playwright-lighthouse for integrated automation
- Accessibility audit is nicely crafted:
 - Best practices, WCAG checks, HTML semantics
 - Visual analysis (colour contrasts, distances, etc.)
- Provides helpful, human readable report



<https://www.npmjs.com/package/playwright-lighthouse>



Using Lighthouse Flow

```
// And Lighthouse flow
const flow = await LighthousePlaywrightFlow
    .startFlow('Airport addition journey', page);

// And take snapshot helper to be passed to each page
const takeLighthouseSnapshot: TakeLighthouseSnapshot = async (name) =>
    await flow.snapshot(name,
        {onlyCategories: Object.keys(lighthouseThresholds), formFactor: 'mobile'});

// ...
// When I perform snapshot analysis for initial view
await addAirportPage.performSnapshotAnalysis('Initial view',
    {takeLighthouseSnapshot, skipAxeViolationChecks: true});

// ...
// Then I generate snapshot report
await flow.generateReports(testInfo, 'lighthouse-test-add-airport-happy-path-flow.html',
    lighthouseThresholds);
```

Creating Lighthouse flow report

```
const res = await this.flow.createFlowResult ();

// Attach Lighthouse report to Playwright report
await testInfo.attach(filename, {body: reports.toString()});

const lighthouseThresholds = {'best-practices': 100, accessibility: 90, seo: 50};

const categories = Object.keys(lighthouseThresholds)
  as ('accessibility' | 'best-practices' | 'seo')[];
let thresholdFailures: string[] = [];

res.steps.forEach((step) => { /* ... */ });

expect(thresholdFailures).toStrictEqual([]);
```

Axe accessibility testing engine

- Static code analysis provided by industry leader
- Produces raw, textual, in-depth reports
- Restricts scanning against certain criterias (i.e. up to WCAG 2.1 AA)



<https://playwright.dev/docs/accessibility-testing>



Using Axe static code analysis

```
private async assertAxeChecks (snapshotTitle: string, skipViolationChecks: boolean) {
    const axeChecks = await new AxeBuilder({page: this.page})
        .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa', 'wcag22aa'])
        .analyze();

    await this.testInfo.attach(this.composeAlllyReportName (snapshotTitle), {
        body: JSON.stringify(axeChecks, null, 2),
        contentType: 'application/json',
    });

    if (!skipViolationChecks) {
        expect(axeChecks.violations).toEqual([]);
    }
}
```

Aria snapshot

- Accessibility tree idea ⁽¹⁾
- It is like a DOM dedicated for a11y
- Playwright Aria snapshots functionality
- Accessibility regression pattern



<https://playwright.dev/docs/aria-snapshots#aria-snapshots>

(1) https://developer.mozilla.org/en-US/docs/Glossary/Accessibility_tree

Aria snapshot regression

```
private async performAriaSnapshotComparison (snapshotTitle: string) {
  expect(await this.page.locator('body').ariaSnapshot()).toMatchSnapshot([
    ARIA_SNAPSHOTS_DIRECTORY,
    `${this.composeSnapshotName(snapshotTitle)}.yaml`,
  ]);
}

private composeSnapshotName (snapshotTitle: string) {
  return slugify(`${this.pageTitle} ${this.testSuite} ${snapshotTitle}`);
}
```

Aria snapshot example

- banner:
 - link "Airports dashboard":
 - img
 - text: Airports dashboard
 - navigation:
 - list:
 - listitem:
 - link "Airports"
 - listitem:
 - link "Add airport"
- main:
 - alert: Sorry, there is some connectivity error
 - paragraph:
 - text: We were unable to fetch some of data necessary to display the page. You can
 - button "restart data fetching"
 - text: manually.
 - paragraph: If it won't help, try to restart the app.

Aria snapshot example



Airports dashboard

Airports **Add airport**

Sorry, there is some connectivity error

We were unable to fetch some of data necessary to display the page. You can [restart data fetching](#) manually.
If it won't help, try to restart the app.

Unknown super-power of aria snapshot

```
- listitem:  
  - link "Airports"  
- listitem:  
  - link "Add airport"  
- main:  
  - heading "Add airport" [level=1]  
  - text: Name  
  - textbox "Name"  
  - text: IATA Code  
  - textbox "IATA Code"  
  - text: Country  
  - combobox "Country": Select one  
  - text: Regions
```

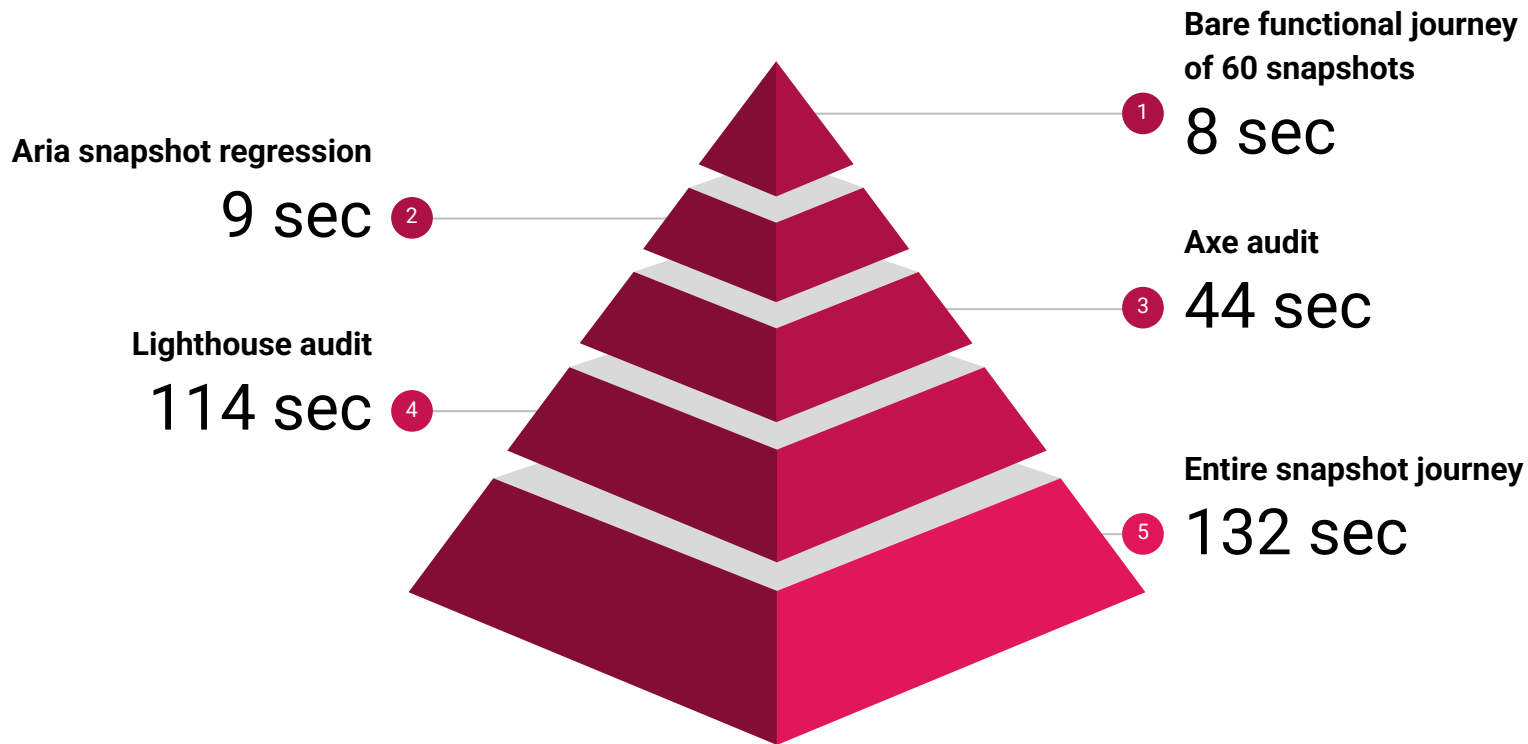
Gotchas and findings

Gotchas and findings

- Snapshot suite is time consuming



Snapshot execution times



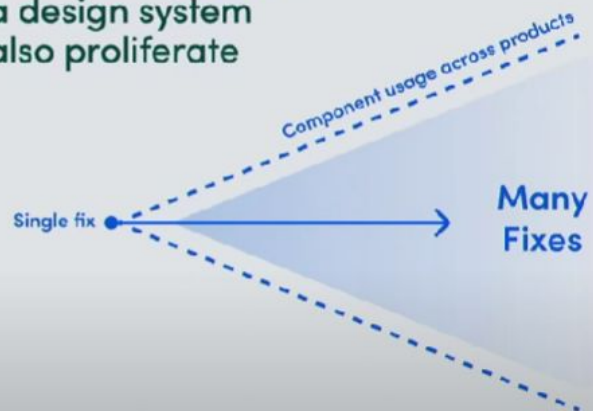
Gotchas and findings

- Snapshot suite is time consuming
- It's crucial to cover a11y on design system





Accessibility fixes in
a design system
also proliferate



16



8:39 / 35:05 • Components >



Auditing Design Systems for Accessibility by Anna E. Cook

Gotchas and findings

- Snapshot suite is time consuming
- It's crucial to cover a11y on design system
- Devs to test their own application, while QAs to focus on entire product



App vs. Product scope

Product section



App section



The screenshot shows a Google search interface with the query "react conferences 2025". The search results are categorized under "All". The first result is from "React" (https://react.dev › community › conferences) titled "React Conferences", describing the "React Universe Conf 2025" in Wrocław, Poland. The second result is from "dev.events" (https://dev.events › react) titled "React conferences 2025 / 2026", listing events like "React Paris 2025" and "react. Wasm I/O". The third result is also from "dev.events" (https://dev.events › Europe) titled "React conferences in Europe 2025 / 2026", listing events like "React conferences in Europe 2025 / 2026" and "The Principal Dev – Masterclass for Tech Leads".

Google react conferences 2025

All News Videos Images Short videos Forums Web More

Usa Florida Europe Best

React
https://react.dev › community › conferences

React Conferences
React Universe Conf 2025. September 2-4, 2025. Wrocław, Poland. Website - Twitter - LinkedIn. React India ...

dev.events
https://dev.events › react

React conferences 2025 / 2026
React Paris 2025 · React conference in Paris, France. 10% discount with code devEvents. Mar 27-28 25. react. Wasm I/O · React conference in Barcelona, Spain.

Europe Poland Spain Germany

dev.events
https://dev.events › Europe

React conferences in Europe 2025 / 2026
React conferences in Europe 2025 / 2026 · Best React conferences 2025 / 2026 · The Principal Dev – Masterclass for Tech Leads · Clean Architecture Masterclass.

Gotchas and findings

- Snapshot suite is time consuming
- It's crucial to cover a11y on design system
- Devs to test their own application, while QAs to focus on entire product
- A11y testing is relatively new thing. It lacks global solutions, toolings or good practice definitions
- Beware of paid solutions



Conclusions

Conclusions

- A11y is not about being perfect. It's about trying.
- So, we did not become an a11y experts. We are, however, an a11y-aware.
- We are able to detect only small factor of a11y errors
- Tests with real users and assistive technologies is the key





Text generation



Diamond



penpot

Diamond



LeoVegas
MOBILE GAMING GROUP

Platinum



Medida

Silver

TV TradingView



<https://www.youtube.com/watch?v=X8wcbJR9laU>

AI and Accessibility: the Good, the Bad, and the Bollocks by Léonie Watson

Playwright disclaimer

DISCLAIMER

Automated accessibility tests can detect some common accessibility problems such as missing or invalid properties. But many accessibility problems can only be discovered through manual testing. We recommend using a combination of automated testing, manual accessibility assessments, and inclusive user testing.

For manual assessments, we recommend [Accessibility Insights for Web](#), a free and open source dev tool that walks you through assessing a website for [WCAG 2.1 AA](#) coverage.

Source: <https://playwright.dev/docs/accessibility-testing>

Q & A

Don't forget the repo
with full a11y tests
suite included.



<https://github.com/LukaszNowakPL/web-a11y-testing>